

Parcial práctico: API REST con Express, autenticación, autorización y despliegue

1. Objetivo de aprendizaje

Desarrollar y desplegar una API REST funcional utilizando Node.js, Express y MongoDB, aplicando buenas prácticas de organización del backend, persistencia de datos, autenticación, autorización por roles, validación de datos y seguridad básica en servicios web.

2. Descripción general del parcial

El estudiante deberá desarrollar una **API REST** utilizando **Node.js, Express y MongoDB**. El tema del proyecto será libre, siempre que el sistema resuelva una necesidad concreta y permita gestionar información mediante operaciones de creación, consulta, actualización y eliminación.

El proyecto debe incluir autenticación de usuarios, autorización por roles, validación de datos, manejo de errores, documentación de endpoints y despliegue en un servidor accesible desde Internet. De ser posible, el sistema deberá estar disponible mediante un dominio o subdominio.

3. Modalidad sugerida

- **Modalidad:** individual o en parejas.
 - **Duración sugerida:** 1 a 2 semanas.
 - **Puntaje:** 100 puntos.
 - **Bonificación opcional:** hasta 5 puntos adicionales por dominio/subdominio funcional con HTTPS.
-

4. Tema del proyecto

El tema será de libre elección. Cada estudiante o equipo debe seleccionar un problema o necesidad que pueda resolverse mediante una API REST.

El tema seleccionado debe permitir evaluar:

1. Gestión de usuarios.
 2. Autenticación.
 3. Autorización por roles.
 4. Gestión de al menos dos entidades principales.
 5. Operaciones CRUD.
 6. Validación de datos.
 7. Persistencia en base de datos.
 8. Despliegue del sistema.
-

5. Condiciones obligatorias del proyecto

5.1 Tema y alcance

Cada proyecto debe tener:

1. Un tema definido.
2. Una breve descripción del problema que resuelve.
3. Objetivo del sistema.
4. Alcance claro del proyecto.
5. Al menos **dos tipos de usuarios o roles**.
6. Al menos **dos entidades principales**, sin contar la entidad usuario.

Ejemplo:

Tema: Sistema de reservas de citas

Entidades:

- Usuario
- Servicio
- Cita

Roles:

- admin
- cliente

5.2 Tecnologías obligatorias

El proyecto debe utilizar:

1. Node.js.
2. Express.
3. MongoDB.
4. Mongoose.
5. JSON Web Token (JWT).
6. Librería para cifrado de contraseñas.
7. Variables de entorno.
8. Postman/Swagger o documentación equivalente para probar la API.

5.3 Requisitos mínimos de la API

La API debe incluir:

1. Inicio de sesión.
2. Cifrado de contraseñas.
3. Generación de token de autenticación.
4. Middleware para validar autenticación.
5. Middleware para validar autorización por roles.

6. CRUD completo de al menos una entidad principal.
7. CRUD parcial o completo de una segunda entidad.
8. Validación de datos de entrada.
9. Manejo centralizado de errores.
10. Respuestas HTTP claras y consistentes.
11. Uso adecuado de códigos de estado HTTP.
12. Documentación de endpoints.
13. Despliegue funcional.

6. Condiciones mínimas de rutas

Cada API debe tener, como mínimo, rutas similares a las siguientes. Los nombres pueden cambiar según el tema seleccionado.

6.1 Autenticación

Método	Ruta sugerida	Descripción	Acceso
POST	/api/auth/register	Registrar usuario	Público
POST	/api/auth/login	Iniciar sesión	Público
GET	/api/auth/profile	Consultar perfil del usuario autenticado	Usuario autenticado

6.2 Entidad principal

Método	Ruta sugerida	Descripción	Acceso sugerido
POST	/api/recurso-principal	Crear registro	Usuario autenticado
GET	/api/recurso-principal	Listar registros	Según rol
GET	/api/recurso-principal/:id	Consultar registro por ID	Según rol
PUT o PATCH	/api/recurso-principal/:id	Actualizar registro	Según rol
DELETE	/api/recurso-principal/:id	Eliminar registro	Admin o rol autorizado

6.3 Segunda entidad

Método	Ruta sugerida	Descripción	Acceso sugerido
POST	/api/segunda-entidad	Crear registro	Usuario autenticado
GET	/api/segunda-entidad	Listar registros	Según rol
GET	/api/segunda-entidad/:id	Consultar registro por ID	Según rol
PATCH	/api/segunda-entidad/:id	Actualizar información específica	Según rol

7. Autenticación y autorización

El sistema debe demostrar que existen rutas públicas y rutas protegidas.

7.1 Rutas públicas

Ejemplos:

```
POST /api/auth/register
POST /api/auth/login
GET /api/public
```

7.2 Rutas protegidas

Ejemplos:

```
GET /api/auth/profile
POST /api/productos
PATCH /api/reservas/:id
DELETE /api/usuarios/:id
```

7.3 Reglas de autorización

El estudiante debe definir permisos según los roles del sistema.

Ejemplo:

Rol	Permisos
admin	Puede gestionar todos los recursos.
usuario	Puede crear y consultar sus propios registros.
moderador / técnico / encargado	Puede revisar, aprobar, actualizar o asignar registros.

Debe existir al menos un caso donde:

1. Un usuario autenticado sí pueda acceder.
 2. Un usuario autenticado no pueda acceder por falta de rol.
 3. Un usuario no autenticado sea rechazado.
-

8. Condiciones adicionales sugeridas

8.1 Diseño y arquitectura

1. Separar el proyecto en rutas, controladores, modelos y middlewares.
 2. Separar la configuración de la base de datos.
 3. Separar la lógica de negocio en servicios.
 4. Usar una estructura de carpetas clara.
 5. Incluir un archivo `server.js` y un archivo `app.js`.
 6. Evitar colocar toda la lógica dentro de un solo archivo.
 7. Incluir un middleware global de errores.
 8. Incluir una ruta de prueba tipo `/api/health`.
-

8.2 Base de datos

1. Usar al menos dos colecciones relacionadas.
 2. Usar referencias entre documentos.
 3. Validar campos obligatorios desde el modelo.
 4. Agregar fechas de creación y actualización.
 5. Evitar eliminar físicamente ciertos registros y usar un campo como `activo`.
 6. Crear datos iniciales de prueba.
 7. Evitar duplicados en campos importantes, por ejemplo correo, código o nombre único.
-

8.3 Seguridad

1. No subir el archivo `.env` al repositorio.
 2. Entregar un archivo `.env.example` sin credenciales reales.
 3. Cifrar contraseñas.
 4. Validar token en rutas privadas.
 5. Validar rol antes de permitir acciones críticas.
 6. No mostrar errores internos del servidor al usuario final.
 7. Configurar CORS si se consume desde otro origen.
 8. Usar Helmet para agregar cabeceras básicas de seguridad.
 9. Limitar intentos o frecuencia de solicitudes en rutas sensibles.
 10. Evitar devolver la contraseña en las respuestas.
-

8.4 Validación

1. Validar campos obligatorios.
 2. Validar formato de correo.
 3. Validar longitud mínima de contraseña.
 4. Validar valores permitidos en campos tipo estado, rol, prioridad o categoría.
 5. Validar que un ID tenga formato correcto antes de consultar la base de datos.
 6. Validar que el recurso exista antes de actualizarlo o eliminarlo.
 7. Responder con mensajes claros cuando los datos enviados sean incorrectos.
-

8.5 Documentación

1. Incluir `README.md` completo.
 2. Explicar cómo instalar el proyecto.
 3. Explicar cómo ejecutar el proyecto localmente.
 4. Listar variables de entorno necesarias.
 5. Documentar cada endpoint.
 6. Indicar qué rutas requieren token.
 7. Indicar qué roles pueden usar cada ruta.
 8. Incluir ejemplos de cuerpo de solicitud.
 9. Incluir ejemplos de respuesta.
 10. Entregar colección de Postman/Swagger.
-

8.6 Pruebas

1. Probar registro de usuario.
 2. Probar inicio de sesión.
 3. Probar acceso con token válido.
 4. Probar acceso sin token.
 5. Probar acceso con rol no autorizado.
 6. Probar creación de un recurso.
 7. Probar actualización de un recurso.
 8. Probar eliminación o desactivación de un recurso.
 9. Probar errores de validación.
 10. Probar la API ya desplegada, no solo localmente.
-

8.7 Despliegue

1. La API debe estar disponible mediante una URL pública.
 2. La base de datos debe estar conectada en producción.
 3. Las variables de entorno deben estar configuradas en el servidor.
 4. El proyecto no debe depender de la computadora local.
 5. El enlace debe estar funcional el día de la evaluación.
 6. Debe existir evidencia de pruebas realizadas en producción.
 7. Implementación de dominio o subdominio.
 8. HTTPS opcional.
-

9. Despliegue

El proyecto debe estar desplegado en un servidor o plataforma accesible desde Internet.

El despliegue debe incluir:

1. URL pública funcional.
2. Conexión real a base de datos.
3. Variables de entorno configuradas en producción.
4. API funcionando sin depender de la computadora local.

10. Documentación requerida

El repositorio debe incluir un archivo **README.md** con:

1. Nombre del proyecto.
2. Descripción breve de la API.
3. Tecnologías utilizadas.
4. Instrucciones para instalación local.
5. Variables de entorno requeridas.
6. Instrucciones para ejecutar el proyecto.
7. Lista de endpoints.
8. Roles disponibles y permisos.
9. Credenciales de prueba o instrucciones para crear usuarios.
10. URL del despliegue.
11. Evidencia o explicación del despliegue realizado.

También deberán entregar una colección de **Postman** o documentación equivalente.

11. Evidencias o entregables esperados

El estudiante debe entregar:

1. Enlace al repositorio.
 2. URL pública de la API desplegada.
 3. Archivo **README.md**.
 4. Archivo **.env.example**.
 5. Colección de Postman, Insomnia o documentación equivalente.
 6. Capturas o evidencias de pruebas:
 - registro;
 - login;
 - acceso con token;
 - acceso denegado por rol;
 - creación de recurso;
 - actualización de recurso;
 - consulta desde producción.
 7. Breve explicación del tema seleccionado.
 8. Breve explicación de roles y permisos.
 9. Evidencia del despliegue.
 10. Evidencia del dominio o subdominio, si aplica.
-

12. Rúbrica de evaluación

Criterio	Puntaje
----------	---------

Criterio	Puntaje
Definición del tema, alcance y entidades	5
Estructura del proyecto y organización del código	10
Modelado de datos con MongoDB/Mongoose	10
Implementación de rutas y operaciones CRUD	10
Autenticación con JSON Web Token (JWT)	15
Autorización por roles	15
Validación de datos y manejo de errores	10
Buenas prácticas de seguridad	5
Documentación y pruebas	5
Despliegue funcional con subdominio o dominio	5
Repositorio en Github/Gitlab o similar	5
Total	100